M09 — Prácticas: despliegue integral de AceriaApp en Docker Desktop para Windows

Autor del curso: JAIME GALLO

Identificación. M09 es exclusivamente práctico: desplegar en Docker todos los componentes ejecutables de AceriaApp, reutilizando el código de M08 sin introducir una implementación C# alternativa. El alumno utiliza **Windows** como sistema operativo anfitrión, **Docker Desktop + WSL 2 con contenedores Linux** como motor y **PowerShell 7** como terminal. Los proyectos de los ocho módulos se mantienen intactos.

1. Qué hace Docker en cada módulo y por qué

Módulo	Contenedor	Para qué	Dónde corre la API	Evidencia
M01	Ninguno	Introducción sin Docker	Windows + LocalDB	120 pasos y E2E SQL/HTTP
M02–M05	SQL Server Express mediante Testcontainers, *opcional*	Bases aisladas para tests de integración	Windows + LocalDB	xUnit, EF Core real, pruebas de API
M06	RabbitMQ con Docker Compose; SQL Server también puede arrancarse en Testcontainers para tests	Comunicación asíncrona real y outbox/inbox	Producción e Inventario en Windows	mensajes publicados, consumidos y persistidos
M07	RabbitMQ heredado de M06	readiness, fallo, recuperación y resiliencia	Windows	health y flujo tras recuperar broker
M08	RabbitMQ heredado; SQL opcional en Testcontainers para tests	Entrega de eventos detrás de YARP y balanceo	Gateway, Producción e Inventario en Windows	HTTP, proyección persistida, balanceo

Tres escenarios distintos:

1. **Sin Docker:** Windows + SQL Server Express LocalDB, para APIs y tests con `ACERIA_TEST_SQLSERVER` definido (M02–M05).
2. **Docker como infraestructura:** Windows sigue ejecutando las APIs mediante `dotnet run`, y Docker aloja RabbitMQ mediante `compose.yml` (M06–M08). Un contenedor en ejecución no certifica que el evento haya llegado.
3. **Todo en Docker:** laboratorio independiente de las prácticas anteriores. Despliega seis contenedores lógicos: SQL Server Express, RabbitMQ, Producción, Inventario 1, Inventario 2 y YARP. Los contenedores son Linux sobre Docker Desktop; el ordenador y la consola siguen siendo Windows.

2. Preparación verificable del equipo Windows

Requisitos: Windows 10/11 compatible con Docker Desktop y virtualización habilitada, PowerShell 7, SDK .NET 8, Visual Studio con ASP.NET Core y suficiente memoria/disco para SQL Server más cinco procesos. Como orientación práctica, reserva al motor Docker varios gigabytes de RAM y al menos 15–

20 GB libres; la cifra efectiva depende del equipo. Para la opción sin Docker instala también SQL Server Express LocalDB.

Instalación y primer diagnóstico (PowerShell 7, ventana de Windows):

```
wsl --status
wsl --update
docker version
docker compose version
docker info --format '{{.OSType}}'
docker run --rm hello-world
dotnet --version
```

Abre **Docker Desktop** y activa el motor **WSL 2 / Linux containers**. La instrucción `docker info --format '{{.OSType}}'` debe devolver `linux`; no significa que el usuario deba instalar Linux como sistema principal. `hello-world` solo prueba que Docker ejecuta una imagen básica; **no** comprueba SQL Server, RabbitMQ ni AceriaApp. Si WSL 2 no está instalado, sigue el asistente oficial de Docker Desktop y reinicia Windows cuando lo solicite. Si `docker version` no comunica con el servidor, no continúes con los ejercicios.

Conceptos antes de ejecutar: una *imagen* es la plantilla inmutable; un *contenedor* es su proceso aislado; *Compose* reúne servicios, redes y volúmenes; un *volumen* conserva SQL y mensajes al reiniciar; el nombre de un servicio Compose es su DNS dentro de la red. `localhost` dentro de un contenedor **no** identifica el PC Windows ni otro servicio.

3. Laboratorio A — SQL Server real para los tests de M02–M05

Este laboratorio **no** despliega la API. Testcontainers 3.10.0 prepara un SQL Server Express temporal y limpia recursos de pruebas. La imagen utilizada por el helper del curso es `mcr.microsoft.com/mssql/server:2022-CU14-ubuntu-22.04`. Se ejecuta en Linux containers sobre Windows.

Desde la **raíz del proyecto terminado** de cada punto, por ejemplo `M02_APIS_REST_ASPNET_CORE/PROYECTO/2.4-documentacion-openapi` (consulta el nombre exacto de la etapa en el repositorio), ejecuta:

```
Remove-Item Env:\ACERIA_TEST_SQLSERVER -ErrorAction SilentlyContinue
docker info --format '{{.OSType}}'
dotnet test .\AceriaApp.sln -c Release
docker ps -a
```

Si `ACERIA_TEST_SQLSERVER` está definida, los tests usan esa conexión en lugar de Testcontainers. La base de prueba es distinta de la base didáctica de la API; comprobar un `dotnet build` no prueba persistencia. Durante la ejecución observa los contenedores con `docker ps` desde una segunda terminal. Comprueba en la salida de xUnit que se ejecutaron y aprobaron todos los tests. Ryuk/Testcontainers gestiona los contenedores temporales; no los elimines manualmente mientras haya pruebas vivas.

Alternativa sin Docker (LocalDB, Windows):

```
sqllocaldb start MSSQLLocalDB
$env:ACERIA_TEST_SQLSERVER = 'Server=(localdb)\MSSQLLocalDB;Database=master;Integrated
Security=True;TrustServerCertificate=True'
dotnet test .\AceriaApp.sln -c Release
```

En ambos casos el proveedor sigue siendo SQL Server. No cambies a SQLite o `InMemory` para fabricar un resultado correcto. Repite la prueba con Docker apagado: la alternativa LocalDB debería funcionar si la instancia está instalada y sana; la variante Testcontainers debe fallar claramente y no aprobar sin motor.

4. Laboratorio B — RabbitMQ con Docker, aplicaciones en Windows

Este es el despliegue **real y mínimo** utilizado desde M06. En el punto 6.1, las prácticas construyen `compose.yml`; M07 y M08 lo heredan. El fichero define `rabbitmq:4.1.5-management`, credenciales didácticas `curso / curso-local`, puertos AMQP `5672` y consola `15672` publicados solo en `127.0.0.1`, y volumen `rabbitmq-datos`.

Desde la carpeta de la etapa (ejemplo final `M08_API_GATEWAY/PROYECTO/8.4-balanceo-carga`):

```
Test-Path .\compose.yml
docker compose -f .\compose.yml config --quiet
docker compose -f .\compose.yml up -d rabbitmq
docker compose -f .\compose.yml ps
docker compose -f .\compose.yml exec -T rabbitmq rabbitmq-diagnostics -q ping
Test-NetConnection localhost -Port 5672
Test-NetConnection localhost -Port 15672
```

El diagnóstico del broker debe devolver código cero, los dos puertos deben estar disponibles y la interfaz `http://localhost:15672/` debe permitir entrar con credenciales **solo de laboratorio**. En **cada** PowerShell en que arranques Producción o Inventario configura:

```
$env:ASPNETCORE_ENVIRONMENT = 'Development'
$env:Comunicacion__RabbitMqUri = 'amqp://curso:curso-local@localhost:5672/'
$env:Comunicacion__WorkersActivos = 'true'
$env:Comunicacion__Prefijo = 'aceria-m08-manual'
```

Completa asimismo las cadenas LocalDB detalladas en M06–M08. Ejecuta `dotnet run --project src/AceriaApp.Inventario.Api --no-launch-profile` y `dotnet run --project src/AceriaApp.Api --no-launch-profile` en terminales distintas, estableciendo puertos y demás variables **del punto correspondiente**. En M08 arranca también `src/AceriaApp.Gateway` y la segunda instancia de Inventario como describe el recorrido 8.4. Las aplicaciones no están en Docker: para ellas `localhost:5672` señala al broker expuesto en el host Windows.

Prueba de fallo real: tras comprobar la entrega outbox/inbox, detén el broker mediante `docker compose stop rabbitmq`, observa readiness degradada y recupera con `docker compose start rabbitmq`. No utilices `down -v` si quieres preservar las colas, las bases o las pruebas de reinicio. `docker compose logs --tail 80 rabbitmq` ayuda a diagnosticar; el estado `Up` por sí solo no acredita mensajería.

5. Laboratorio C — despliegue integral de M08 con Docker Compose

Propósito: aprender la diferencia entre usar Docker solo como proveedor de infraestructura y contenerizar Producción, Inventario y YARP. Este laboratorio reutiliza **sin cambiar** la etapa `M08_API_GATEWAY/PROYECTO/8.4-balanceo-carga`. Los componentes reales son:

- `sqlserver`: SQL Server 2022 Express, puerto **1433 privado** en la red Compose, volumen `sql-datos`.
- `rabbitmq`: AMQP, gestión web y volumen `rabbit-datos`.
- `inventario-1` y `inventario-2`: dos instancias del proyecto `AceriaApp.Inventario.Api` sobre **una misma base**.
- `produccion`: `AceriaApp.Api` con las bases aisladas de API/Hexagonal/DDD y su publicador outbox.
- `gateway`: `AceriaApp.Gateway` (YARP), único puerto público de negocio: `127.0.0.1:5100`.

Atención al cambio técnico: `(localdb)\MSSQLLocalDB` es solo Windows y no funciona dentro de estos contenedores Linux. Por eso, en este laboratorio se conectan las aplicaciones a `Server=sqlserver,1433` con autenticación SQL de laboratorio; esta diferencia se realiza mediante variables

`ConnectionStrings__*` de Compose **sin modificar el código C#**. En la red Docker, `rabbitmq:5672` reemplaza `localhost:5672` y los destinos YARP pasan a `http://inventario-1:8080/`, `inventario-2` y `produccion`. El código sigue apuntando a los mismos contratos, no a nuevos ejemplos.

5.1 Preparar parámetros y construir las imágenes

Desde la raíz del repositorio, entra en `M09_DESPLIEGUE_DOCKER\PROYECTO`:

```
Set-Location .\M09_DESPLIEGUE_DOCKER\PROYECTO
Copy-Item .\.env.example .\.env
notepad .\.env
docker compose --env-file .\.env -f .\docker\compose.fullstack.yml config --quiet
.\EJECUTAR_DOCKER_WINDOWS.ps1 -Accion preparar
```

Cambia la clave del ejemplo de `.env` por otra robusta y **no publiques ese archivo**. La conexión SQL tiene cifrado activado, pero confía en el certificado autofirmado solo para laboratorio. `.env` se usa en Compose; no sustituye a los archivos de configuración del proyecto.

El `Dockerfile.Servicio` compila con `mcr.microsoft.com/dotnet/sdk:8.0`, publica el proyecto con `dotnet publish` y lo ejecuta sobre `mcr.microsoft.com/dotnet/aspnet:8.0`, sin SDK en la imagen final. El argumento de construcción selecciona `AceriaApp.Api`, `AceriaApp.Inventario.Api` o `AceriaApp.Gateway`. El contexto es la **raíz del repositorio**, por lo que están disponibles todas las referencias entre proyectos.

5.2 Iniciar paso a paso y ejecutar pruebas HTTP

```
.\EJECUTAR_DOCKER_WINDOWS.ps1 -Accion levantar
```

El script inspecciona Docker, valida Compose, arranca primero SQL y RabbitMQ, espera a que SQL responda de verdad y levanta **secuencialmente** Inventario 1, Inventario 2, Producción y Gateway. Esta secuencia evita que las dos instancias creen simultáneamente las tablas de Inventario. Después comprueba readiness de los backends, liveness del gateway, consulta de stock, crea una orden DDD de dos líneas, planifica por HTTP, exige `202`, exige `409` al repetir, espera la proyección asíncrona y contrasta los datos después de reiniciar los procesos. Cualquier diferencia debe devolver error, no un falso PASS.

Repite las comprobaciones sin reconstruir:

```
.\EJECUTAR_DOCKER_WINDOWS.ps1 -Accion verificar
docker compose --env-file .\.env -f .\docker\compose.fullstack.yml ps
docker compose --env-file .\.env -f .\docker\compose.fullstack.yml logs --tail 100 produccion inventario-1
inventario-2 gateway
```

Las respuestas de `/health/ready` validan dependencias (SQL y AMQP); `/health/live` del gateway solo valida el proceso. Un `200` de liveness no demuestra que YARP pueda llegar a Inventario. El stock `Acero` de una base nueva debe ser 300 y los eventos llegan de forma asíncrona, por lo que hay que esperar y consultar su proyección.

5.3 Persistencia, recuperación y limpieza controlada

Los datos SQL y mensajes RabbitMQ sobreviven a `restart` y `stop` gracias a los volúmenes. El script comprueba la recuperación de un identificador concreto. Para detener sin borrar:

```
.\EJECUTAR_DOCKER_WINDOWS.ps1 -Accion detener
```

Para eliminar **contenedores y red** pero conservar datos:

```
.\EJECUTAR_DOCKER_WINDOWS.ps1 -Accion bajar
```

Solo para **reiniciar de cero los datos didácticos** (destrucción intencional de ambos volúmenes):

```
.\EJECUTAR_DOCKER_WINDOWS.ps1 -Accion limpiar -ConfirmarBorradoDatos
```

Nunca ejecutes esa última operación cuando desees demostrar persistencia o reentrega. El broker no debe reutilizarse simultáneamente con el Compose de M06–M08 en el mismo puerto 5672; elige un entorno activo y detén el otro antes de comenzar.

5.4 Taller dirigido: comprender, construir y ejecutar los seis contenedores

El apartado anterior ofrece el procedimiento completo. Esta ampliación explica **cada decisión técnica** para que el alumno sea capaz de reconstruir el despliegue sin memorizar comandos. Se trabaja con PowerShell 7 sobre **Windows**; los contenedores usan Linux internamente gracias a Docker Desktop y WSL2.

Paso 9.1 — Identificar cada proceso y su ubicación física

Antes de ejecutar, dibuja esta distribución:

```
PC del alumno: Windows + PowerShell + Visual Studio + Docker Desktop
  localhost:5100  -> Gateway (YARP)
  localhost:5105  -> Producción
  localhost:5106  -> Inventario 1
  localhost:5107  -> Inventario 2
  localhost:5672  -> RabbitMQ (AMQP)
  localhost:15672 -> RabbitMQ Management
           |
           v  Puertos publicados únicamente sobre 127.0.0.1
Red privada de Docker, llamada "aceria"
  gateway:8080     -> YARP
  produccion:8080  -> API y publicador outbox
  inventario-1:8080-> Consumidor y API
  inventario-2:8080-> Consumidor y API
  rabbitmq:5672    -> Broker
  sqlserver:1433   -> SQL Server Express
```

Para qué: el nombre de servicio es un DNS interno de Docker. En Producción, `localhost` significa «el contenedor Producción», no el equipo Windows ni el broker. Por eso la configuración del laboratorio emplea `rabbitmq:5672` y `inventario-1:8080`; el navegador del alumno sí utiliza `localhost` y los puertos publicados.

Verificación: abre el Compose y encuentra seis entradas bajo `services`. Identifica el único contenedor que usa YARP y comprueba que SQL Server no publica un puerto al anfitrión.

Paso 9.2 — Comprender qué se construye con el Dockerfile

Abre `docker/Dockerfile.Servicio`. Su primera etapa utiliza la imagen `mcr.microsoft.com/dotnet/sdk:8.0` y las instrucciones `COPY`, `dotnet restore` y `dotnet publish`. Esa etapa resuelve dependencias y compila el **código real de M08**. La segunda etapa utiliza `mcr.microsoft.com/dotnet/aspnet:8.0` y copia la salida publicada; no necesita instalar Visual Studio ni el SDK para ejecutar la API.

`ARG PROYECTO` permite reutilizar el mismo Dockerfile para `AceriaApp.Api`, `AceriaApp.Inventario.Api` o `AceriaApp.Gateway`. Cada servicio produce su propia imagen de aplicación. `ASPNETCORE_URLS=http://+:8080` hace que Kestrel escuche dentro del contenedor; `EXPOSE 8080` documenta el puerto, pero **no lo publica automáticamente** a Windows.

Acción:

```
Set-Location .\M09_DESPLIEGUE_DOCKER\PROYECTO
docker compose --env-file .\.env -f .\docker\compose.fullstack.yml build produccion
if ($LASTEXITCODE -ne 0) { throw 'Falló el build real de Producción.' }
docker image ls
```

Resultado esperado: aparece una imagen para Producción y la salida de compilación muestra que el proyecto `AceriaApp.Api` se publicó sobre .NET 8. Si falla un `ProjectReference`, diagnostica la ruta del contexto de construcción; no elimines proyectos heredados ni cambies el dominio para maquillar la compilación.

Paso 9.3 — Diferenciar imagen, contenedor y servicio Compose

Imagen: resultado reutilizable de una compilación. **Contenedor:** proceso aislado creado a partir de la imagen. **Servicio Compose:** declaración de cómo construir, configurar, comunicar y reiniciar uno o varios contenedores. Los dos inventarios tienen el mismo proyecto y usan **el mismo puerto interno 8080**, pero dos nombres y dos puertos diferentes en Windows.

```
docker compose --env-file .\.env -f .\docker\compose.fullstack.yml config --services
docker compose --env-file .\.env -f .\docker\compose.fullstack.yml ps
```

La primera orden lista **seis servicios declarados**; la segunda solo presenta contenedores creados. Un servicio presente en el YAML no demuestra que esté ejecutándose, y un contenedor que indica `Up` no demuestra que su API ni su base de datos funcionen.

Paso 9.4 — Seguir las variables de configuración hasta ASP.NET Core

Las variables de `build.args` se usan **durante la compilación**; las variables de `environment` se inyectan **cuando arranca el proceso**. En ASP.NET Core, una doble barra baja en el nombre de variable representa una sección: `ConnectionStrings__AceriaProduccion` sustituye a `ConnectionStrings:AceriaProduccion` en la configuración.

Localiza estas cinco variables en el Compose y anota el servicio destinatario, su propósito y el error esperado si se modifica incorrectamente:

1. `ConnectionStrings__AceriaProduccion`: conexión del contexto DDD y outbox a SQL Server.
2. `ConnectionStrings__AceriaInventario`: ambas instancias apuntan a la misma base relacional.
3. `Comunicacion__RabbitMqUri`: comunica productores y consumidores con el broker mediante AMQP.
4. `Comunicacion__InventarioUrl`: servicio interno de Inventario para peticiones HTTP de Producción.
5. `ReverseProxy__Clusters__inventario-cluster__Destinations__inventario-1__Address`: destino YARP por nombre interno.

La configuración de los proyectos originales con `(localdb)\MSSQLLocalDB` sigue siendo correcta **cuando la API corre en Windows**. Dentro de un contenedor Linux no existe LocalDB; aquí se sustituye la conexión por `Server=sqlserver,1433`, sin cambiar las clases ni el proveedor EF Core SQL Server. **Nunca se debe describir LocalDB como un servicio que corre dentro del contenedor.**

Paso 9.5 — Comprender redes, puertos y volúmenes

En `compose.fullstack.yml`, `networks: aceria` proporciona resolución de nombres internos. `ports` establece un puente de **un puerto de Windows hacia el puerto 8080 del contenedor**, por ejemplo `127.0.0.1:5106:8080`. El primer número es el puerto del ordenador; el último es el puerto de Kestrel. SQL Server usa el puerto privado `1433`, no expuesto a Windows.

Los volúmenes `sql-datos` y `rabbit-datos` mantienen los datos de SQL Server y RabbitMQ aunque se reinicien los procesos. El almacenamiento persistente **no** depende de que el contenedor de la API conserve su sistema de archivos.

Comprueba antes y después de arrancar:

```
docker network ls
docker volume ls
docker compose --env-file .\.env -f .\docker\compose.fullstack.yml ps
```

No utilices `docker volume prune` ni `docker compose down -v` mientras debas demostrar persistencia. Si eliminas los volúmenes, puedes volver a construir un laboratorio vacío, pero habrás destruido las pruebas de conservación de datos del recorrido anterior.

Paso 9.6 — Crear contraseñas de laboratorio y evitar exponer secretos

`.env.example` proporciona un valor **solo demostrativo**. Cópialo como `.env` y modifica `ACERIA_SA_PASSWORD` antes de levantar SQL Server. El archivo real `.env` está ignorado por Git y no debe enviarse con el repositorio. El usuario RabbitMQ `curso` y su clave didáctica son también exclusivamente para pruebas locales. El Compose limita al loopback `127.0.0.1` los puertos publicados.

```
Copy-Item .\.env.example .\.env
notepad .\.env
git status --short
```

El archivo `.env` no debe aparecer como nuevo archivo listo para commit. No pegues en informes compartidos el resultado completo de `docker compose config` si contiene valores de configuración sensibles. La opción `TrustServerCertificate=True` de este laboratorio **no representa una configuración de TLS adecuada para producción**.

Paso 9.7 — Entender las comprobaciones de salud reales

Un contenedor SQL en estado `running` todavía puede estar recuperando una base. El `healthcheck` consulta el motor SQL y evita que las APIs se inicien antes de que esté preparado. RabbitMQ usa `rabbitmq-diagnostics -q ping`. Los cuatro procesos .NET tienen comprobaciones de **readiness**, mientras `/health/live` de YARP solo acredita que el gateway está vivo.

```
docker compose --env-file .\.env -f .\docker\compose.fullstack.yml ps
docker compose --env-file .\.env -f .\docker\compose.fullstack.yml exec -T rabbitmq rabbitmq-diagnostics -q ping
Invoke-RestMethod 'http://localhost:5100/health/live'
Invoke-RestMethod 'http://localhost:5105/health/ready'
Invoke-RestMethod 'http://localhost:5106/health/ready'
Invoke-RestMethod 'http://localhost:5107/health/ready'
```

Resultado: la respuesta del gateway indica que vive y los tres backends muestran `Healthy`. Una respuesta incorrecta, incluso con todos los contenedores en estado `Up`, significa que las dependencias no están listas y debe diagnosticarse con los logs del servicio afectado.

5.5 Taller dirigido: API Gateway, mensajería y resiliencia

En los siguientes pasos, el alumno **no cambia código de negocio**: ejecuta el sistema que ya construyó con las prácticas M02–M08 y comprueba por HTTP los mismos contratos de AceriaApp.

Paso 9.8 — Comprobar que YARP realmente utiliza las dos instancias

```
$gateway = 'http://localhost:5100'
$instancias = @{}
foreach ($i in 1..16) {
  $respuesta = Invoke-WebRequest "$gateway/api/v1/materiales/Acero"
  if ($respuesta.StatusCode -ne 200) { throw "Error HTTP $($respuesta.StatusCode)" }
  $nombre = [string]$respuesta.Headers['X-Instancia']
  if ($nombre) { $instancias[$nombre] = $true }
```

```
}
if (-not $instancias.ContainsKey('inventario-1') -or
    -not $instancias.ContainsKey('inventario-2')) {
  throw "No se observaron las dos instancias: $($instancias.Keys -join ', ')"
}
```

Qué se prueba: el proxy YARP reparte solicitudes entre destinos sanos y reenvía el contrato HTTP sin crear un inventario falso. Las dos respuestas deben consultar el mismo SQL Server y ofrecer el mismo stock. RoundRobin reparte solicitudes, no garantiza el mismo tiempo de trabajo para cada instancia.

Paso 9.9 — Crear una orden persistente mediante la API real

```
$cuerpo = @{ cliente='Axia Windows Docker'; material='Acero'; cantidad=20 } | ConvertTo-Json
$creada = Invoke-WebRequest -Uri "$gateway/api/v1/produccion/ordenes" -Method Post -ContentType 'application/json' -
Body $cuerpo -SkipHttpErrorCheck
if ($creada.StatusCode -ne 201) { throw "POST crear: $($creada.StatusCode)" }
$ordenId = [string](($creada.Content | ConvertFrom-Json).id)
$ordenUri = "$gateway/api/v1/produccion/ordenes/$ordenId"
$linea = @{ material='Inoxidable'; cantidad=10 } | ConvertTo-Json
$agregada = Invoke-WebRequest -Uri "$ordenUri/lineas" -Method Post -ContentType 'application/json' -Body $linea -
SkipHttpErrorCheck
if ($agregada.StatusCode -ne 200) { throw "POST línea: $($agregada.StatusCode)" }
$borrador = Invoke-RestMethod $ordenUri
if ($borrador.estado -ne 'Borrador' -or $borrador.cantidadTotal -ne 30) {
  throw 'El agregado SQL no conserva el total y el estado esperados.'
}
```

Para qué: se comprueba el endpoint DDD, el contrato JSON, las dos líneas del agregado y la persistencia en SQL Express. Conserva `$ordenId` y `$ordenUri` durante el ejercicio; generar una orden nueva tras un fallo **no sustituye** la comprobación de la original.

Paso 9.10 — Comprobar que planificar emite un evento y prohíbe una segunda transición

```
$plan = "$gateway/api/v1/comunicacion/ordenes/$ordenId/planificar"
$primera = Invoke-WebRequest -Uri $plan -Method Post -ContentType 'application/json' -Body '{}' -SkipHttpErrorCheck
if ($primera.StatusCode -ne 202) { throw "Se esperaba 202: $($primera.Content)" }
$segunda = Invoke-WebRequest -Uri $plan -Method Post -ContentType 'application/json' -Body '{}' -SkipHttpErrorCheck
if ($segunda.StatusCode -ne 409) { throw "Se esperaba 409: $($segunda.Content)" }
```

Interpretación: `202` significa que se ha aceptado la transición de estado y la escritura de un evento outbox pendiente. **No** significa que el mensaje ya haya sido recibido por Inventario. El `409` protege la transición de una segunda planificación; no sustituye la deduplicación de mensajes en el receptor.

Paso 9.11 — Acreditar la entrega asíncrona en el SQL de Inventario

```
$proyeccionUri = "$gateway/api/v1/planificaciones/$ordenId"
$proyeccion = $null
foreach ($intento in 1..75) {
  $respuesta = Invoke-WebRequest $proyeccionUri -SkipHttpErrorCheck
  if ($respuesta.StatusCode -eq 200) {
    $proyeccion = $respuesta.Content | ConvertFrom-Json
    break
  }
  if ($respuesta.StatusCode -ne 404) { throw "HTTP inesperado: $($respuesta.StatusCode)" }
  Start-Sleep -Seconds 2
}
if ($null -eq $proyeccion -or [string]$proyeccion.ordenId -ne $ordenId -or
    $proyeccion.cantidadTotal -ne 30) {
  throw 'No se acreditó el consumo outbox/inbox.'
}
$eventoId = [string]$proyeccion.eventoId
```

Por qué: la consulta devuelve `404` mientras el evento no haya llegado; cuando el consumidor lo registra, responde `200` con la proyección SQL de la misma orden y el ID del evento. Una cola vacía **no es una evidencia suficiente**: podría indicar entrega correcta o ausencia de publicaciones. El GET de la proyección desambigua el resultado.

Paso 9.12 — Interpretar las colas y los logs de RabbitMQ

Abre `http://localhost:15672/` con las credenciales didácticas. Localiza conexiones, canales y colas del prefijo `aceria-docker-integral`. Identifica el productor outbox y al menos un consumidor. Una vez procesados, los mensajes pueden desaparecer de la cola sin que se pierda su resultado SQL.

```
docker compose --env-file .\.env -f .\docker\compose.fullstack.yml logs --tail 100 rabbitmq
docker compose --env-file .\.env -f .\docker\compose.fullstack.yml logs --tail 100 produccion
docker compose --env-file .\.env -f .\docker\compose.fullstack.yml logs --tail 100 inventario-1
```

Explica por qué la topología de intercambio/cola permite desacoplar temporalmente productor y consumidor. Si la proyección no aparece, comprueba `WorkersActivos`, el mismo `Prefijo`, las credenciales AMQP y los logs; no insertes registros SQL de forma manual.

Paso 9.13 — Simular la caída de Inventario 2 y la recuperación de YARP

```
docker compose --env-file .\.env -f .\docker\compose.fullstack.yml stop inventario-2
Start-Sleep -Seconds 10
foreach ($intento in 1..10) {
  $respuesta = Invoke-WebRequest "$gateway/api/v1/materiales/Acero" -SkipHttpErrorCheck
  if ($respuesta.StatusCode -ne 200 -or [string]$respuesta.Headers['X-Instancia'] -ne 'inventario-1') {
    throw 'YARP no seleccionó el destino sano.'
  }
}
docker compose --env-file .\.env -f .\docker\compose.fullstack.yml start inventario-2
```

Qué se demuestra: tras la convergencia de sus comprobaciones de salud, YARP sirve el recurso mediante el destino superviviente. Repite el Paso 9.8 cuando `5107/health/ready` vuelva a responder `200`. No reinicies el gateway para forzar la recuperación: eso ocultaría una posible deficiencia de detección de destinos.

Paso 9.14 — Detener RabbitMQ sin borrar mensajes ni bases

```
docker compose --env-file .\.env -f .\docker\compose.fullstack.yml stop rabbitmq
$degradada = Invoke-WebRequest 'http://localhost:5106/health/ready' -SkipHttpErrorCheck
Write-Host "Inventario readiness sin broker: $($degradada.StatusCode)"
docker compose --env-file .\.env -f .\docker\compose.fullstack.yml start rabbitmq
docker compose --env-file .\.env -f .\docker\compose.fullstack.yml exec -T rabbitmq rabbitmq-diagnostics -q ping
```

Un proceso de aplicación puede seguir vivo aunque la dependencia AMQP esté caída. El volumen RabbitMQ se conserva. La recuperación del puerto `5672` por sí sola **no acredita reentrega**: repite el flujo de una orden nueva y exige de nuevo una proyección. Para escenarios más profundos de duplicados y reintentos consulta los tests E2E del M06.

Paso 9.15 — Reiniciar aplicaciones y demostrar persistencia real

```
docker compose --env-file .\.env -f .\docker\compose.fullstack.yml restart inventario-1 inventario-2 produccion
gateway
Start-Sleep -Seconds 10
$ordenDespues = Invoke-RestMethod "$gateway/api/v1/produccion/ordenes/$ordenId"
$eventoDespues = Invoke-RestMethod "$gateway/api/v1/planificaciones/$ordenId"
if ($ordenDespues.estado -ne 'Planificada' -or $ordenDespues.cantidadTotal -ne 30) {
  throw 'La orden original no persistió.'
}
if ([string]$eventoDespues.eventoId -ne $eventoId) {
```

```
  throw 'La proyección original no conserva su ID.'
}
```

Si las APIs aún no responden después del reinicio, ejecuta primero los health checks del Paso 9.7. No crees una nueva orden para reparar la prueba. La evidencia exige recuperar **el mismo identificador** de SQL, no un objeto recreado en memoria.

Paso 9.16 — Detener con conservación o destruir datos conscientemente

```
.\EJECUTAR_DOCKER_WINDOWS.ps1 -Accion detener
# Conservar volúmenes, eliminar únicamente contenedores y red:
.\EJECUTAR_DOCKER_WINDOWS.ps1 -Accion bajar
# Solo tras terminar y archivar las pruebas; DESTRUYE datos:
.\EJECUTAR_DOCKER_WINDOWS.ps1 -Accion limpiar -ConfirmarBorradoDatos
```

Justificación: `stop` conserva procesos apagados y sus volúmenes; `down` retira contenedores y red, sin borrar los volúmenes; `down -v` destruye los volúmenes de este proyecto Compose. La última opción es intencionalmente destructiva y nunca se debe utilizar para «arreglar» una prueba fallida de persistencia.

5.6 Matriz de evidencias de la práctica integral

Resultado esperado	Demostración exigida
Docker Desktop Windows sobre WSL2	Windows anfitrión, `docker info` informa `OSType=linux`
Construcción .NET 8 sin copiar binarios previos	`docker compose build` con `Dockerfile.Servicio` y restauración NuGet
SQL Server Express de verdad	servicio `sqlserver` healthy y API con datos recuperados después del reinicio
RabbitMQ operativo	`rabbitmq-diagnostics ping` y consumo de evento real
Gateway funcional	`/health/live`, stock HTTP y rutas proxy
Balanceo real	cabecera `X-Instancia` observada en las dos instancias
Producción DDD	creación HTTP 201, dos líneas, total 30
Outbox e inbox	planificación HTTP 202, proyección 200 y mismo `ordenId`
Segunda planificación prohibida	HTTP 409
Fallo de un destino	tráfico sigue funcionando por Inventario 1 y se recupera Inventario 2
Persistencia	orden y evento conservan IDs después de reiniciar
Reporte reproducible	`.qa/docker-windows-e2e.json` con `resultado=PASS` y plataforma Windows

Entrega el JSON del script, una tabla de resultados de las pruebas manuales y las explicaciones de conexiones entre servicios. **No compartas tu .env real** ni admitas un `docker ps` como sustituto de la prueba SQL/HTTP/AMQP. El profesor marcará `PENDIENTE WINDOWS+DOCKER` hasta ejecutar efectivamente el despliegue integral sobre Windows con Docker Desktop y conservar la evidencia.

6. Diagnóstico orientado a causas, no a atajos

Síntoma	Causa probable	Acción verificable
`docker version` no conecta	Docker Desktop parado o motor equivocado	Arrancar Docker Desktop; comprobar `OSType=linux`

Síntoma	Causa probable	Acción verificable
puerto `5672` ocupado	dos laboratorios RabbitMQ activos	`docker ps --format "table {{.Names}}\t{{.Ports}}"` y detener el que sobra
SQL no llega a `healthy`	memoria, imagen o credenciales; herramientas SQL ausentes	`docker compose logs sqlserver` y revisar prueba de salud; no sustituirlo por InMemory
contenedor arranca y cae	error .NET al inicializar SQL	`docker compose logs produccion` / `inventario-1` y verificar SQL healthy
API dentro de Docker apunta a `(localdb)`	no se configuró `ConnectionStrings__*`	comprobar `docker compose config` sin publicar contraseña
error RabbitMQ desde contenedor	`localhost` en vez de DNS `rabbitmq`, usuarios/puertos	revisar `Comunicacion__RabbitMqUri` y `rabbitmq-diagnostics ping`
gateway devuelve `503`	destinos sin salud o YARP aún sin resolver	verificar `/health/ready` en cada backend y variables ReverseProxy
el evento no aparece	workers desactivados, prefijo distinto o outbox pendiente	revisar `WorkersActivos`, `Prefijo`, broker, logs y proyección
`dotnet test` pasa sin SQL real	test sustituido por proveedor falso	comprobar proveedor SQL Server, evidencias y ausencia de tests omitidos

Criterio de cierre: el Markdown se contrasta con las imágenes, los procesos reales y la suite; Windows LocalDB, Windows+Docker infra y Windows+Docker integral son **tres certificaciones diferenciadas**. El flujo Docker integral solo se marcará PASS tras ejecutar realmente `EJECUTAR_DOCKER_WINDOWS.ps1 -Accion levantar` en un Windows con Docker Desktop/WSL2. Haber creado este manual o superado CI Linux/Windows sin Docker Desktop **no sustituye** esa prueba física.